

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій



Лабораторно робота №11
з дисципліни: “Інтеграція інформаційних систем”
на тему:
“Використання агента Jenkins”

Виконав:
ст. групи ПП-44
ВЕРЕЩАК Богдан

Прийняла:
доц. Оксана ОБОРСЬКА

Львів – 2026

Мета роботи

Навчитись використовувати зв'язку Github + Jenkins + SonarQube + PostgreSQL, прописувати інтеграцію SonarQube в Jenkins, github в pipeline job, та аналізувати результат роботи зв'язки.

Завдання

Завданням даної лабораторної роботи є використання зв'язки Github+Jenkins+SonarQube+PostgreSQL. Бажаний CI процес виглядає наступним чином:

0. Зміни були введені в github репозиторій проекту
1. Jenkins шляхом опитування виявляє зміни, та автоматично тригерить конфігурацію.
2. Проект із змінами вивантажується на Jenkins сервер
3. Jenkins проводить скан проекту та відсилає проект на SonarQube
4. SonarQube дає відповідь чи дані було збережено успішно в PostgreSQL чи ні, а також результат проходження Quality Gate
5. Jenkins проводить процес побудови незалежно від результату Quality Gate.

Блок налаштування GITHUB trigger

1. Перейдіть на веб інтерфейс інструменту Jenkins
2. Зайдіть в конфігурацію Pipeline роботи зробленої в ЛР №4 або ЛР №10.
3. Метою уникнення накладань рекомендується створити окремий Pipeline та про скопіювати сам код Pipeline в новий.
3. В блоці “General” поставте галочку біля опції “GitHub project”.
4. Впишіть адресу Вашого Github репозиторію в текстове поле.
5. Далі перейдіть до блоку “Build Triggers”, та оберіть бажаний тригер.
6. Для варіанту “Poll SCM” потрібно прописати періодичність, з якою Jenkins буде перевіряти репозиторій на предмет змін.
7. Перейдіть в меню “Manage Jenkins”
8. Після встановлення плагіну, перейдіть в меню “Manage Jenkins”

Блок підготовки SonarQube

9. Перейдіть на веб інтерфейс SonarQube
10. Перейдіть на вкладку “Administration”
11. Створіть новий вебхук натисканням на кнопку “Create”
12. Перейдіть в SonarQube на свій профіль
13. Перейдіть на веб інтерфейс інструменту Jenkins
14. Зайдіть в меню “Manage Jenkins” “Configure System”. Знайдіть там блок “SonarQube servers” та додайте SonarQube, натиснувши відповідну кнопку.
15. В даному прикладі перевіряється проект на NodeJS.
16. Оскільки в прикладі використовуватиметься команда “sudo” користувачу, під яким Jenkins виконує дії у себе локально (jenkins) потрібно надати право використовувати “sudo” без паролю.

Блок налаштування та запуску Pipeline Job

17. Якщо ви використовуєте Pipeline з ЛР №10, позабирайте непотрібні кроки
18. Додайте stage “SonarQube Analysis”, або власна назва
19. Спробуйте запустити пайплайн.
20. Можна помітити що команда сканеру значно менша ніж це було в ЛР про SonarQube.
21. Протягом часу зазначеного в періодичності на кроці 6, білд повинен запуститись автоматично, так як було додано новий файл в корінь проекту.
22. Зробіть скріншот подібний до рисунку нижче, а також скопіюйте увесь “Console Output” до кінця виконання роботи сканера, та включіть їх у звіт.

Хід роботи

Plugins

- Updates
- Available plugins
- Installed plugins
- Advanced settings

Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity

SonarQube Scanner Pending

Loading plugin extensions Pending

→ [Go back to the top page](#)

(you can start using the installed plugins right away)

→ ☐ Restart Jenkins when installation is complete and no jobs are running

Рис. 1. Завантаження плагіна SonarQube

B Bohdan

Profile Security Notifications Projects

Name	Type	Project	Last use	Created	Expiration	
jenkins-integration	Project	Book4u Analysis	Never	February 25, 2026	March 26, 2026	Revoke

Рис. 2. Створення токена для інтеграції jenkins

Credentials

+ Add Credentials

.env	System - Global - .env	
github-token	System - Global - github-token	
jenkins-agent/***** (agent-ssh-key)	System - Global - agent-ssh-key - agent-ssh-key	
sonar-token	System - Global - sonar-token	

Рис. 3. Створення даних для sonar

Jenkins / Manage Jenkins / System

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

SonarQube

Server URL

Default is http://localhost:9000

https://sonar.vereshchak.online

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonar-token

+ Add

Advanced

Save

Apply

Рис. 4. Налаштування SonarQube в Jenkins

Jenkins / Manage Jenkins / Tools

SonarQube Scanner installations Edited

+ Add SonarQube Scanner

SonarQube Scanner

Name

SonarQube Scanne

☒ Install automatically ?

Install from Maven Central

Version

SonarQube Scanner 8.0.1.6346

+ Add Installer

Рис. 5. Налаштування інсталятора SonarQube

Jenkins / All / New Item

New Item

Enter an item name

Book4u-CI

Select an item type

Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline

OK

Рис. 6. Створення нового пайплайну

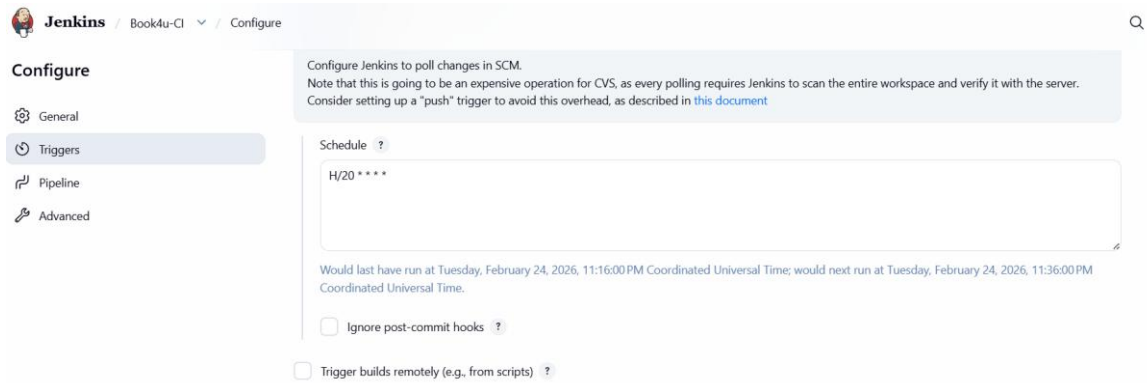


Рис. 7. Налаштування частоти виконання пайплайну

Код пайплайну:

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
            credentialsId: 'github-token',
            url: 'https://github.com/BogdanVereshchak/Book4u'
      }
    }
    stage('SonarQube Analysis') {
      steps {
        script {
          def scannerHome = tool 'SonarQube Scanner'

          withSonarQubeEnv('SonarQube') {
            sh "${scannerHome}/bin/sonar-scanner"
          }
        }
      }
    }
  }

  stage('Quality Gate') {
    steps {
      timeout(time: 1, unit: 'HOURS') {
        waitForQualityGate abortPipeline: true
      }
    }
  }
}
```

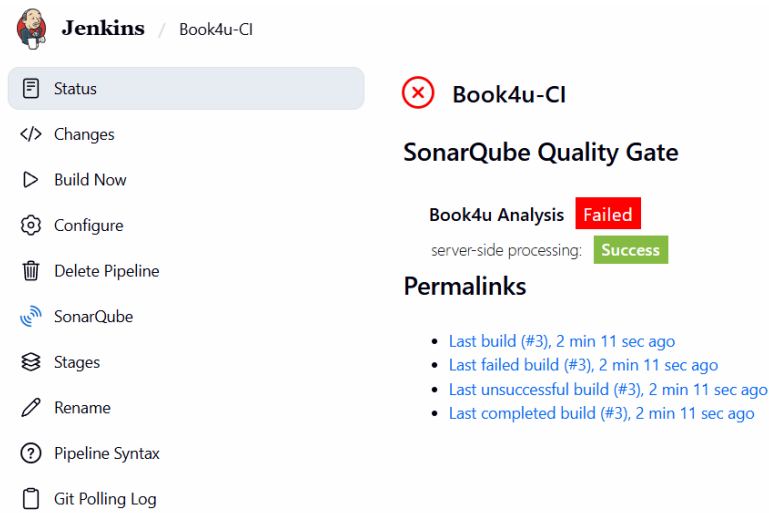


Рис. 8. Результат виконання пайплайну. Успішна збірка, але не пройдена перевірка якості коду

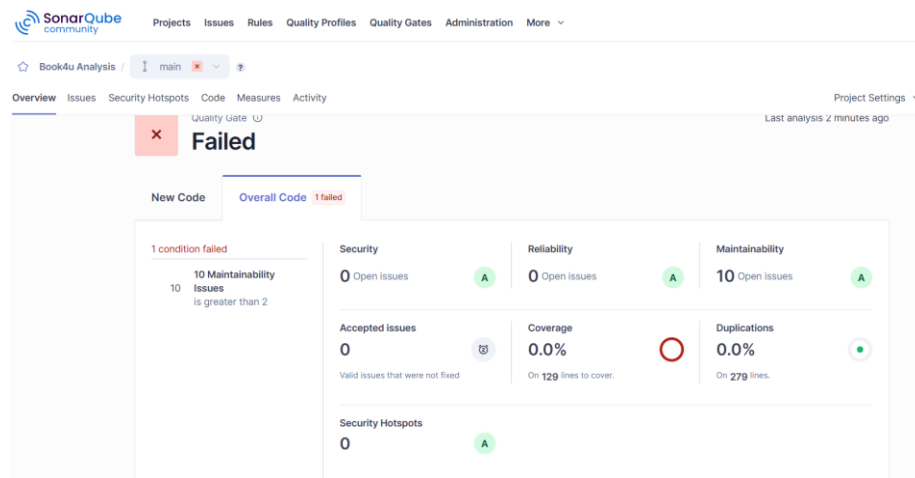


Рис. 9. Результат перевірки якості коду

Висновок

На даній лабораторній роботі я навчився використовувати зв'язку Github + Jenkins + SonarQube + PostgreSQL, прописувати інтеграцію SonarQube в Jenkins, github в pipeline job, та аналізувати результат роботи зв'язки.